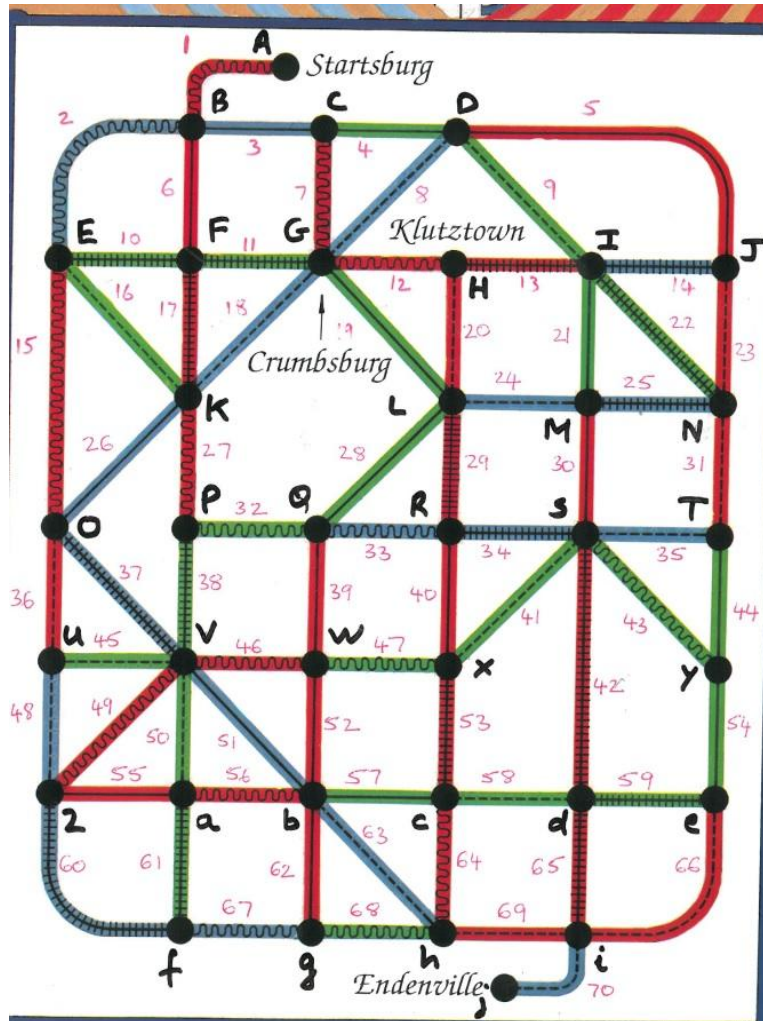


GrandpaTransitMapANNOTATED



با توجه به اینکه عکس ورودی کیفیت لازم رو نداشت و OCR به خوبی عمل نمیکرد، مجبور شدم دستی داده ها از عکس استخراج کنم.

این داده ها شامل حروف دو سر یال و رنگ یال و وزن و نوع وسیله نقلیه آن است.

داده های پروژه

```
edges = [  
  (0, 1, 'red', 1, 'cable'),  
  (1, 2, 'blue', 3, 'trolley'),  
  (2, 3, 'green', 4, 'trolley'),  
  (3, 8, 'green', 9, 'bus'),  
  (8, 9, 'blue', 14, 'horse'),  
  (9, 13, 'red', 23, 'bus'),  
  (2, 6, 'blue', 7, 'trolley'),  
  (6, 10, 'red', 11, 'cable'),  
  (10, 16, 'green', 17, 'bus'),  
  (16, 17, 'blue', 18, 'trolley'),  
  (17, 18, 'red', 19, 'cable'),  
  (18, 19, 'green', 20, 'bus'),  
  (19, 20, 'blue', 21, 'trolley'),  
  (20, 21, 'red', 22, 'cable'),  
  (21, 22, 'green', 23, 'bus'),  
  (22, 23, 'blue', 24, 'trolley'),  
  (23, 24, 'red', 25, 'cable'),  
  (24, 25, 'green', 26, 'bus'),  
  (25, 26, 'blue', 27, 'trolley'),  
  (26, 27, 'red', 28, 'cable'),  
  (27, 28, 'green', 29, 'bus'),  
  (28, 29, 'blue', 30, 'trolley'),  
  (29, 30, 'red', 31, 'cable'),  
  (30, 31, 'green', 32, 'bus'),  
  (31, 32, 'blue', 33, 'trolley'),  
  (32, 33, 'red', 34, 'cable'),  
  (33, 34, 'green', 35, 'bus'),  
  (34, 35, 'blue', 36, 'trolley'),  
  (35, 36, 'red', 37, 'cable'),  
  (36, 37, 'green', 38, 'bus'),  
  (37, 38, 'blue', 39, 'trolley'),  
  (38, 39, 'red', 40, 'cable'),  
  (39, 40, 'green', 41, 'bus'),  
  (40, 41, 'blue', 42, 'trolley'),  
  (41, 42, 'red', 43, 'cable'),  
  (42, 43, 'green', 44, 'bus'),  
  (43, 44, 'blue', 45, 'trolley'),  
  (44, 45, 'red', 46, 'cable'),  
  (45, 46, 'green', 47, 'bus'),  
  (46, 47, 'blue', 48, 'trolley'),  
  (47, 48, 'red', 49, 'cable'),  
  (48, 49, 'green', 50, 'bus'),  
  (49, 50, 'blue', 51, 'trolley'),  
  (50, 51, 'red', 52, 'cable'),  
  (51, 52, 'green', 53, 'bus'),  
  (52, 53, 'blue', 54, 'trolley'),  
  (53, 54, 'red', 55, 'cable'),  
  (54, 55, 'green', 56, 'bus'),  
  (55, 56, 'blue', 57, 'trolley'),  
  (56, 57, 'red', 58, 'cable'),  
  (57, 58, 'green', 59, 'bus'),  
  (58, 59, 'blue', 60, 'trolley'),  
  (59, 60, 'red', 61, 'cable'),  
  (60, 61, 'green', 62, 'bus'),  
  (61, 62, 'blue', 63, 'trolley'),  
  (62, 63, 'red', 64, 'cable'),  
  (63, 64, 'green', 65, 'bus'),  
  (64, 65, 'blue', 66, 'trolley'),  
  (65, 66, 'red', 67, 'cable'),  
  (66, 67, 'green', 68, 'bus'),  
  (67, 68, 'blue', 69, 'trolley'),  
  (68, 69, 'red', 70, 'cable')  
]
```



```
(19, 18, 'blue', 35, 'bus'),  
(17, 18, 'blue', 34, 'horse'),  
(17, 16, 'blue', 33, 'cable'),  
(4, 5, 'green', 10, 'horse'),  
(5, 6, 'green', 11, 'horse'),  
(4, 10, 'green', 16, 'bus'),  
(8, 13, 'green', 22, 'horse'),  
(8, 12, 'green', 21, 'trolley'),  
(6, 11, 'green', 19, 'trolley'),  
(11, 16, 'green', 28, 'trolley'),  
(15, 16, 'green', 32, 'cable'),  
(21, 15, 'green', 38, 'horse'),  
(21, 20, 'green', 45, 'bus'),  
(21, 26, 'green', 50, 'bus'),  
(31, 26, 'green', 61, 'horse'),  
(32, 33, 'green', 68, 'cable'),  
(27, 28, 'green', 57, 'trolley'),  
(28, 29, 'green', 58, 'bus'),  
(30, 29, 'green', 59, 'horse'),  
(30, 50, 'green', 54, 'trolley'),  
(18, 50, 'green', 43, 'cable'),  
(19, 50, 'green', 44, 'trolley'),  
(18, 49, 'green', 41, 'bus'),  
(22, 49, 'green', 47, 'cable') ]
```

سپس جهت جلوگیری از خطای فردی در جمع آوری داده ها از شکل، با استفاده از کد زیر آن ها را بر اساس وزن از یک تا هفتاد مرتب کردم و سپس با استفاده از کد دیگری بررسی کردم که آیا وزنی جا افتاده یا نه.

```
# مرتب‌سازی بر اساس وزن
sorted_edges = sorted(edges, key=lambda x: x[3])

for edge in sorted_edges:
    print(edge)
```

```
# استخراج وزن‌ها از لیست مرتب‌شده
weights = [edge[3] for edge in sorted_edges]

# تولید اعداد مورد انتظار از 1 تا 70
expected_weights = set(range(1, 71))

# تبدیل وزن‌های موجود به مجموعه برای مقایسه
actual_weights = set(weights)

# پیدا کردن شماره‌های جاافتاده
missing_weights = expected_weights - actual_weights

if missing_weights:
    print("وزن‌های جاافتاده:", sorted(missing_weights))
else:
    print("هیچ وزنی جا نیفتاده است.")
```

سپس جهت استفاده راحتتر از داده ها و استفاده از کلاس آن ها را به عدد تبدیل کردیم.

```
connections = [  
    (0, 1, 'r', 'c'), (1, 2, 'b', 't'), (1, 4, 'b', 'c'), (1, 5, 'r', 't'),  
    (2, 3, 'g', 't'), (2, 6, 'r', 'c'), (3, 6, 'b', 'b'), (3, 8, 'g', 'b'),  
    (3, 9, 'r', 't'), (4, 5, 'g', 'h'), (4, 10, 'g', 'b'), (4, 14, 'r', 'c'),  
    (5, 6, 'g', 'h'), (5, 10, 'r', 'h'), (6, 7, 'r', 'c'), (6, 10, 'b', 'b'),  
    (6, 11, 'g', 't'), (7, 8, 'r', 'h'), (7, 11, 'r', 'b'), (8, 9, 'b', 'h'),  
    (8, 12, 'g', 't'), (8, 13, 'g', 'h'), (9, 13, 'r', 'b'), (10, 14, 'b', 't'),  
    (10, 15, 'r', 'c'), (11, 12, 'b', 'b'), (11, 16, 'g', 't'), (11, 17, 'r', 'h'),  
    (12, 13, 'b', 'h'), (12, 18, 'r', 't'), (13, 19, 'r', 'b'), (14, 20, 'r', 'b'),  
    (14, 21, 'b', 'h'), (15, 16, 'g', 'c'), (15, 21, 'g', 'h'), (16, 17, 'b', 'c'),  
    (16, 22, 'r', 't'), (17, 18, 'b', 'h'), (17, 23, 'r', 't'), (18, 19, 'b', 'b'),  
    (18, 23, 'g', 'b'), (18, 24, 'g', 'c'), (18, 29, 'r', 'h'), (19, 24, 'g', 't'),  
    (20, 21, 'g', 'b'), (20, 25, 'b', 'b'), (21, 22, 'r', 'c'), (21, 25, 'r', 'c'),  
    (21, 26, 'g', 'b'), (21, 27, 'b', 't'), (22, 23, 'g', 'c'), (22, 27, 'r', 't'),  
    (23, 28, 'r', 'h'), (24, 30, 'g', 't'), (25, 31, 'b', 'h'), (25, 26, 'r', 't'),  
    (26, 27, 'r', 'c'), (26, 31, 'g', 'h'), (27, 28, 'g', 't'), (27, 32, 'r', 't'),  
    (27, 33, 'b', 'b'), (28, 29, 'g', 'b'), (28, 33, 'r', 'c'), (29, 30, 'g', 'h'),  
    (29, 34, 'r', 'h'), (30, 34, 'r', 'b'), (31, 32, 'b', 'c'), (32, 33, 'g', 'c'),  
    (33, 34, 'r', 'b'), (34, 35, 'b', 'b')  
]
```

```
class Road:
    def __init__(self, company='.', road_type='.'):
        self.company = company
        self.type = road_type

class Node:
    def __init__(self, village, age, parent=None):
        self.parent = parent
        self.children = []
        self.village = village
        self.age = age + 1
        self.done = False

    def add_children(self):
        self.done = True
        for i in range(36):
            if grid[self.village][i].type != '.' and grid[self.village][i].company
            != '.':
                if unexplored[self.village][i] > 0:
                    child = Node(i, self.age, self)
                    self.children.append(child)
                    unexplored[self.village][i] -= 1

        for child in self.children:
            if not child.done:
                child.add_children()

    def find_path_to(self, target, path=None):
        if path is None:
            path = []

        path.append(self.village)

        if self.village == target:
            return path

        for child in self.children:
            result = child.find_path_to(target, path.copy())
```

```
        return result

    return None

grid = [[Road() for _ in range(36)] for _ in range(36)]
unexplored = [[25 for _ in range(36)] for _ in range(36)]
villages = [chr(65 + i) for i in range(36)]
alldone = False

for i, j, company, road_type in connections:
    grid[i][j] = Road(company, road_type)
    grid[j][i] = grid[i][j]

startsburg = Node(0, 0)
startsburg.add_children()

path_to_35 = startsburg.find_path_to(35)

if path_to_35:
    print("Path to node 35:", path_to_35)
else:
    print("No path to node 35 found.")
```

[illegible]

Graph Representation with Edge Colors and Labels

